

untangle::executor

Generated by Doxygen 1.18.0

1 executor	1
2 Namespace Index	5
2.1 Namespace List	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Namespace Documentation	11
5.1 untangle Namespace Reference	11
5.1.1 Detailed Description	11
6 Class Documentation	13
6.1 untangle::executor< taskT > Class Template Reference	13
6.1.1 Detailed Description	13
6.1.2 Constructor & Destructor Documentation	14
6.1.2.1 executor()	14
6.1.2.2 ~executor()	14
6.1.3 Member Function Documentation	15
6.1.3.1 add_task()	15
6.1.3.2 stop()	15
6.1.4 Member Data Documentation	15
6.1.4.1 on_task_error	15
7 File Documentation	17
7.1 executor.hpp	17
Index	21

Chapter 1

executor

c++ thread pool executor based on async execution

The interface is one header that exposes a pool of workers, each one an **async** execution in continuous mode, fed by a shared queue of tasks.

This is the prototype, imported. It was written in `async/prototypes/` to answer whether execution is a usable building block for a pool — it is — and it is being hardened here. The behaviour below is what the tests state today, not a settled interface.

requirements

C++23, and the `async` submodule:

```
git submodule update --init --recursive
```

`async` carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with `std::println`, so they need a standard library that provides `<print>` — GCC 14 or newer.

example

```
#include <executor.hpp>
#include <functional>

int main() {
    untangle::executor<std::function<void(void)>> pool(4);

    for (int i = 0; i < 100; ++i) {
        pool.add_task([i] { work(i); });
    }

    return 0; // ~executor finishes what was submitted, then stops the workers
}
```

the task type

The pool is a template on the task it runs, the way an execution is a template on its action. One pool runs one signature, and the caller names it:

```
untangle::executor<std::function<int(void)>> pool(4); // returns are run and dropped
```

The task type does not have to be a `std::function`. What an execution requires of it is a nested `result_<_type`, which it reads rather than deduces, so any callable declaring one will do — and a pool built on such a callable never touches `std::function::result_type`, which the standard removed in C++20 and both `libc++` and `libstdc++` still supply by grace rather than contract.

```
struct my_task {
    using result_type = void;
    void operator()() const;
};
```

```
untangle::executor<my_task> pool(4);
```

A task type that takes arguments is refused, with a `static_assert` rather than an error inside `std::bind`: the pool queues tasks, and has nowhere to keep arguments for them until a worker frees up.

how work is placed

N executions run in continuous mode behind a shared `std::deque` of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue finished — on `_finished`, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. `execution::is_busy()` is what `add_task()` reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

lifetime and failure

The destructor finishes what was submitted. It refuses further work, waits for the queue to empty and for every worker to go idle, and only then stops them. A task submitted before the pool goes out of scope has run by the time it does.

`add_task()` says whether the task was taken. It returns false once the pool has stopped accepting — which a task adding more work from inside the pool can see — and false again if the worker it was offered to has stopped, which is what `stop()` leaves behind. A refused task is destroyed rather than queued, so a false answer means it will not run.

`stop()` stops the workers. What they are running they finish; what is still queued stays there unrun, and anything added afterwards is refused. It does not wait for the workers to leave their threads — only the destructor does that, and that wait is what makes destroying the pool safe.

A task that throws does not take the worker with it, and what it threw is not lost. The worker catches it and carries on with the next task; `on_task_error` is where the throw goes:

```

untangle::executor<std::function<void(void)» pool(4);

pool.on_task_error = [](std::exception_ptr thrown) {
    try {
        std::rethrow_exception(thrown);
    } catch (const std::exception& e) {
        log(e.what());
    }
};

```

It carries a `std::exception_ptr` because a task may throw something that is not a `std::exception`, and that is the case most worth hearing about. Assign it before the first `add_task()` — a worker reads it — and expect it on a worker's thread, with more than one worker possibly inside it at once. Leave it unset and the pool warns on `stderr` instead, naming the worker.

A return value is still not collected: a continuous worker keeps none, so a task with something to report carries its own channel.

building the tests

```

cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build

```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access.

sanitizers

`EXECUTOR_SANITIZE` builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```

cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan

```

Accepted values are `address`, `thread`, `undefined`, or `empty`. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. `address` and `thread` instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

`tools/make_doc.sh` builds `doc/refman.pdf` from the header and this README. It needs `doxygen`, `tectonic` and `python3`, and it fails on any `doxygen` warning that is not an `obsolete-tag` notice, so an undocumented member or a broken `\ref` does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	A thread pool executor built out of <code>untangle::async::execution</code> objects	11
--------------------------	---	--------------------

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::executor< taskT >	
Runs tasks on a fixed pool of untangle::async::execution workers	13

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp	17
--	----

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of `untangle::async::execution` objects.

Classes

- class `executor`
Runs tasks on a fixed pool of `untangle::async::execution` workers.

5.1.1 Detailed Description

A thread pool executor built out of `untangle::async::execution` objects.

Imported from the `async` repo's `prototypes/executor.hpp`, unchanged apart from its namespace and this comment. The hardening the prototype's README asks for has not been done yet.

The pool holds `N` executions in continuous mode and a shared queue of tasks. A task is handed straight to a free worker only when the shared queue is empty; otherwise it goes to the back of the queue, and a worker takes the front of the queue as it frees up. Nothing overtakes: finding a free worker does not let a task jump a queue that already has work in it.

Chapter 6

Class Documentation

6.1 `untangle::executor< taskT >` Class Template Reference

Runs tasks on a fixed pool of `untangle::async::execution` workers.

```
#include <executor.hpp>
```

Public Member Functions

- `executor` (`std::size_t worker_count`)
Starts `worker_count` executions in continuous mode.
- `~executor` ()
Finishes what has been submitted, then stops every worker.
- `bool add_task` (`taskT task`)
Queues a task, or hands it to a worker that has nothing to do.
- `std::size_t pending` () `const`
How many tasks are waiting for a worker.
- `void stop` ()
Stops every worker. What they are already running, they finish.

Public Attributes

- `std::function< void(std::exception_ptr)>` `on_task_error`
Called with whatever a task threw. Assigned by the caller.

6.1.1 Detailed Description

```
template<typename taskT>  
class untangle::executor< taskT >
```

Runs tasks on a fixed pool of `untangle::async::execution` workers.

Template Parameters

taskT	The type of the task the pool runs, specified as <code>std::function<...></code> like the execution it is handed to. One pool runs one signature, because an execution's action type is fixed by its own template argument.
-------	---

Remarks

A task that needs to return something carries its own channel: a continuous worker does not collect results, so a non-void return is run and dropped.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 executor()

```
template<typename taskT>
untangle::executor< taskT >::executor (
    std::size_t worker_count) [inline], [explicit]
```

Starts `worker_count` executions in continuous mode.

Parameters

<code>worker_count</code>	- How many workers to run. Must not be zero.
---------------------------	--

Exceptions

<code>std::invalid_argument</code>	- <code>worker_count</code> is zero. Such a pool would take work and never run it, so it is refused before any worker starts rather than left to hang in the destructor.
------------------------------------	--

Remarks

`std::thread::hardware_concurrency()` returns 0 when it cannot tell how many cores there are, and a caller passing that straight through is the likely way in. Choosing a default then is the caller's call, not the pool's.

6.1.2.2 ~executor()

```
template<typename taskT>
untangle::executor< taskT >::~executor () [inline]
```

Finishes what has been submitted, then stops every worker.

Both waits are unbounded - a pool that abandoned a running task would be worse than one that waits for it - and both report on `stderr` rather than stalling in silence, naming the workers they are still waiting on.

Remarks

Every worker has left its thread before the first execution is destroyed. `~execution()` waits only for its own worker, so destroying them one at a time would free the first while another's thread was still reading it through `nothing_running()`; the poll waits for all of them. It is this pool's own poll, so a second pool running alongside it is not waited for.

6.1.3 Member Function Documentation

6.1.3.1 add_task()

```
template<typename taskT>
bool untangle::executor< taskT >::add_task (
    taskT task) [inline]
```

Queues a task, or hands it to a worker that has nothing to do.

Returns

true - the task was accepted. false - it was refused, because the pool has stopped accepting or because the worker it was offered to has stopped. Either way it will not run.

6.1.3.2 stop()

```
template<typename taskT>
void untangle::executor< taskT >::stop () [inline]
```

Stops every worker. What they are already running, they finish.

A worker stops taking work from the moment this returns, so anything still in the queue stays there unrun, and a task added afterwards is refused by the worker it is handed to. The destructor calls this itself, after waiting for the queue to empty - which is the difference between stopping a pool and finishing one.

Remarks

Calling it twice is harmless: the second `stop()` finds a worker that has already stopped and does nothing.

Attention

It does not wait for the workers to leave their threads. Only the destructor does that, and it is what makes destroying the pool safe rather than this.

6.1.4 Member Data Documentation

6.1.4.1 on_task_error

```
template<typename taskT>
std::function<void(std::exception_ptr)> untangle::executor< taskT >::on_task_error
```

Called with whatever a task threw. Assigned by the caller.

The pool runs the caller's code and nothing comes back from it: `add_task()` has answered true long before the task runs, and a task that failed is otherwise indistinguishable from one that succeeded. This is the one way out. Without it the only record is a warning on stderr naming a worker the caller neither chose nor can look up.

Remarks

It receives a `std::exception_ptr` because a task may throw something that is not a `std::exception`, and that case is the one most worth hearing about. Rethrow it to read it.

Attention

It runs on a worker's thread, and more than one worker may be in it at once - whatever it touches has to be safe for that. Assign it before the first [add_task\(\)](#): a worker reads it, so assigning it while tasks are running is a data race.

Nothing may escape it. It is called from inside the worker's own catch, and an exception leaving a detached thread function calls `std::terminate` - `async` catches around it to keep that from being fatal, but the exception is then lost.

The documentation for this class was generated from the following file:

- `executor.hpp`

Chapter 7

File Documentation

7.1 executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00014 #pragma once
00015
00016 #include <algorithm>
00017 #include <async.hpp>
00018 #include <chrono>
00019 #include <condition_variable>
00020 #include <cstdint>
00021 #include <deque>
00022 #include <exception>
00023 #include <functional>
00024 #include <memory>
00025 #include <mutex>
00026 #include <print>
00027 #include <stdexcept>
00028 #include <string>
00029 #include <thread>
00030 #include <type_traits>
00031 #include <utility>
00032 #include <vector>
00033
00034 namespace untangle {
00035
00046 template <typename taskT>
00047 class executor {
00048     // add_action() is the only way in, and the pool calls it with no arguments to bind: what it
00049     // queues is the task itself, and there is nowhere to keep arguments alongside it until a worker
00050     // is free. Stated here so a pool on std::function<void(int)> says why rather than failing inside
00051     // std::bind.
00052     static_assert(std::is_invocable_v<taskT>,
00053         "executor: the task type must be callable with no arguments");
00054
00055 public:
00068     explicit executor(std::size_t worker_count) {
00069         if (worker_count == 0) {
00070             throw std::invalid_argument("executor: worker_count must not be zero");
00071         }
00072
00073         workers_.reserve(worker_count);
00074
00075         for (std::size_t index = 0; index < worker_count; ++index) {
00076             auto next = worker_execution::create_instance("pool_worker_" + std::to_string(index));
00077
00078             // How a worker learns there is more waiting for it. is_busy() answers whether a worker has
00079             // room; this is what tells the pool the moment one frees up, without anybody polling.
00080             next->on_finished = [this, index] { take_next_task(index); };
00081
00082             // What a task throws, on its way to whoever submitted it. The worker catches it and would
00083             // otherwise print a warning naming itself, which reaches a log and no code.
00084             //
00085             // Assigned whether or not a caller has set on_task_error, because it cannot be assigned later
00086             // - a worker thread reads it, and this is the last moment nothing is running. The name is
00087             // captured rather than looked up for the same reason: report_task_error() runs on the
00088             // worker's thread and taking mutex_ there to read workers_ would be a lock this path does not
00089             // need.
00090             next->on_error = [this, name = next->name](std::exception_ptr thrown) {
```

```

00091     report_task_error(name, thrown);
00092 };
00093
00094 // A worker runs in a detached thread and cannot be joined, so asking whether it is still
00095 // running is the only way to wait for one, and the poll answers that for any number of them
00096 // at once. A poll answers for the executions added to it, and this one is added the pool's
00097 // own workers and nothing else, so what the destructor waits for is exactly these. Nothing
00098 // here has to unregister - the record runs both ways, so whichever of the two dies first
00099 // takes itself out of the other.
00100 poll_.add(*next);
00101
00102 workers_.push_back(std::move(next));
00103 workers_.back()->start();
00104 }
00105 }
00106
00107 ~executor() {
00108 {
00109     std::unique_lock<std::mutex> lock(mutex_);
00110     accepting_ = false;
00111
00112     // Unbounded, and reported rather than bounded. Both waits below keep waiting for as long as
00113     // it takes; what they stop doing is keeping it to themselves.
00114     auto interval = std::chrono::milliseconds(report_first_ms);
00115     auto waited = std::chrono::milliseconds(0);
00116
00117     while (!finished_cv_.wait_for(lock, interval,
00118         [this] { return pending_.empty() && nothing_running(); })) {
00119         waited += interval;
00120
00121         // The lock is held here, which is what makes both reads consistent with the predicate that
00122         // just failed. is_busy() takes each execution's action_mutex_ under this one, which is the
00123         // same order give_to_worker() uses - no new lock order is introduced by reporting.
00124         std::println(
00125             stderr, "executor: still waiting to finish after {}s - {} queued, {} in a task{}",
00126             waited.count() / 1000, pending_.size(), count_workers(&worker_execution::is_busy),
00127             name_workers(&worker_execution::is_busy));
00128
00129         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00130     }
00131 }
00132 }
00133
00134 // The workers are stopped through the same call a caller would make, after the wait above has
00135 // left nothing for them to do.
00136 stop();
00137
00138 // The poll stands in for a join. Until it reports idle, a worker can still be inside
00139 // take_next_task(), reading every execution through nothing_running() and is_busy() - which is
00140 // what makes destroying them here a use-after-free rather than a teardown.
00141 //
00142 // A different question from the finish above, and reported as one: that wait is about workers
00143 // still inside a task, this is about workers whose thread has not left. A worker can be idle
00144 // and still be here.
00145 auto interval = std::chrono::milliseconds(report_first_ms);
00146 auto waited = std::chrono::milliseconds(0);
00147
00148 while (poll_.is_running()) { // time of check
00149     const auto tick = std::chrono::milliseconds(poll_interval_ms);
00150     std::this_thread::sleep_for(tick);
00151     waited += tick;
00152
00153     if (waited >= interval) {
00154         std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00155             waited.count() / 1000, count_workers(&worker_execution::is_running),
00156             name_workers(&worker_execution::is_running));
00157
00158         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00159     }
00160 }
00161 }
00162
00163 // time of use: no worker thread is left to reach workers_, so clearing it races nothing.
00164 workers_.clear();
00165 }
00166
00167 bool add_task(taskT task) {
00168     std::lock_guard<std::mutex> lock(mutex_);
00169
00170     if (!accepting_) {
00171         return false;
00172     }
00173
00174     // The queue comes first whenever it has anything in it: a task that arrives while others are
00175     // waiting does not get to overtake them just because a worker happens to be free.
00176     if (pending_.empty()) {
00177         for (std::size_t index = 0; index < workers_.size(); ++index) {
00178             if (!workers_[index]->is_busy()) {
00179                 // A refusal is not a reason to offer it to the next worker, and it cannot be queued

```

```

00196         // instead: add_action() has already destroyed the task. It means this worker is stopped,
00197         // and a stopped worker is one stop() stopped - so the rest are stopped too, and false is
00198         // the answer for all of them.
00199         return give_to_worker(index, std::move(task));
00200     }
00201 }
00202 }
00203
00204     pending_.push_back(std::move(task));
00205     return true;
00206 }
00207
00211     std::size_t pending() const {
00212         std::lock_guard<std::mutex> lock(mutex_);
00213         return pending_.size();
00214     }
00215
00230     void stop() {
00231         // Outside any lock: stop() wakes each worker, and a worker waking up runs on_finished, which
00232         // wants mutex_. workers_ is not written after the constructor, so reading it here races
00233         // nothing - only the destructor clears it, and only after the poll says no worker is left.
00234         for (auto& one : workers_) {
00235             one->stop();
00236         }
00237     }
00238
00258     std::function<void(std::exception_ptr)> on_task_error;
00259
00260 private:
00268     void report_task_error(const std::string& worker, std::exception_ptr thrown) {
00269         if (on_task_error) {
00270             on_task_error(thrown);
00271             return;
00272         }
00273
00274         // Nobody is listening, so this stands in for the warning async would have printed had the pool
00275         // not taken its on_error. Losing it silently would make a pool worse than the execution it
00276         // wraps.
00277         try {
00278             std::rethrow_exception(thrown);
00279         } catch (const std::exception& e) {
00280             std::println(stderr, "warning: executor task on '{}' threw: {}", worker, e.what());
00281         } catch (...) {
00282             std::println(stderr, "warning: executor task on '{}' threw an unknown type", worker);
00283         }
00284     }
00285
00286     using worker_execution = untangle::async::execution<taskT>;
00287
00301     [[nodiscard]] bool give_to_worker(std::size_t index, taskT task) {
00302         // add_action() takes the execution's own action_mutex while this holds mutex_. That is only
00303         // safe in one direction, and it holds: a worker calls on_finished with action_mutex released,
00304         // so it never takes mutex_ while holding action_mutex, and the two never form a cycle.
00305         //
00306         // The answer is checked rather than dropped. Until stop() was a caller's to call this could not
00307         // be false - a worker refuses only once stopped, and only the destructor stopped one, after the
00308         // queue was empty - so an unchecked answer cost nothing. It costs a lost task now.
00309         return workers_[index]->add_action(std::move(task));
00310     }
00311
00315     void take_next_task(std::size_t index) {
00316         std::lock_guard<std::mutex> lock(mutex_);
00317
00318         if (!pending_.empty()) {
00319             taskT next = std::move(pending_.front());
00320             pending_.pop_front();
00321
00322             if (!give_to_worker(index, std::move(next))) {
00323                 // Not put back, because there is nothing left to put back. Reported instead: add_task()
00324                 // told this task's submitter it had been taken, and that has just stopped being true.
00325                 std::println(stderr, "executor: {} refused a queued task, which is lost",
00326                     workers_[index]->name);
00327             }
00328
00329             return;
00330         }
00331
00332         if (nothing_running()) {
00333             finished_cv_.notify_all();
00334         }
00335     }
00336
00343     bool nothing_running() const {
00344         for (const auto& one : workers_) {
00345             if (one->is_busy()) {
00346                 return false;
00347             }

```

```
00348     }
00349
00350     return true;
00351 }
00352
00353 std::size_t count_workers(bool (worker_execution::*state)() const) const {
00354     std::size_t count = 0;
00355
00356     for (const auto& one : workers_) {
00357         if ((one.get()->*state)()) {
00358             ++count;
00359         }
00360     }
00361
00362     return count;
00363 }
00364
00365 std::string name_workers(bool (worker_execution::*state)() const) const {
00366     std::string names;
00367
00368     for (const auto& one : workers_) {
00369         if ((one.get()->*state)()) {
00370             names += names.empty() ? " : " : ", ";
00371             names += one->name;
00372         }
00373     }
00374
00375     return names;
00376 }
00377
00378 static constexpr int poll_interval_ms = 50;
00379
00380 static constexpr int report_first_ms = 1000;
00381 static constexpr int report_max_ms = 30000;
00382
00383 mutable std::mutex mutex_;
00384 std::condition_variable finished_cv_;
00385
00386 async::execution_poll poll_;
00387
00388 std::deque<taskT> pending_;
00389 std::vector<std::shared_ptr<worker_execution>> workers_;
00390 bool accepting_ = true;
00391 };
00392
00393 // namespace untangle
```

Index

- ~executor
 - untangle::executor< taskT >, [14](#)
- add_task
 - untangle::executor< taskT >, [15](#)
- executor, [1](#)
 - untangle::executor< taskT >, [14](#)
- on_task_error
 - untangle::executor< taskT >, [15](#)
- stop
 - untangle::executor< taskT >, [15](#)
- untangle, [11](#)
- untangle::executor< taskT >, [13](#)
 - ~executor, [14](#)
 - add_task, [15](#)
 - executor, [14](#)
 - on_task_error, [15](#)
 - stop, [15](#)